

基于Qwen3.x 看Ascend 950上如何进行推理优化

作者: Wayne, 小E

时间: 2026.9.11

目录

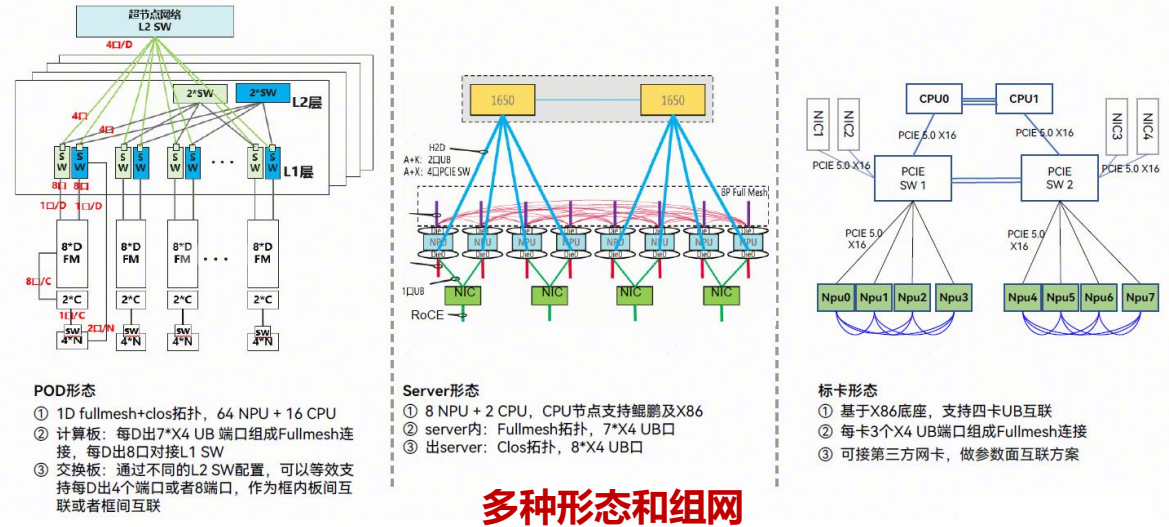
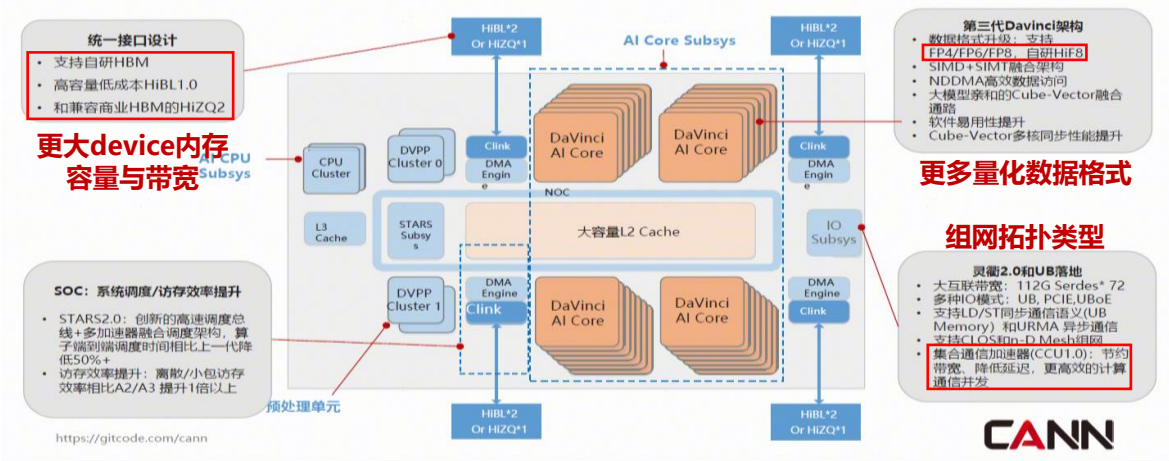
Part 1 Ascend 950硬件概览

Part 2 优化前置分析

Part 3 Qwen3.5优化点实施

Part 4 总结

Ascend 950硬件概览



规格项		昇腾950PR	昇腾950DT
算力	Cube算力规格	378T (FP16/BF16)	432~486T (FP16/BF16)
		757T (MxFP8, HiF8, MxFP6)	864~972T (MxFP8, HiF8, MxFP6)
		1514T (MxFP4)	1728~1944T (MxFP4)
	Vector算力规格 (FP16/BF16)	47T	54~60.8T
Memory	总算力规格 (FP16/BF16)	425T	486~547T
	Memory容量 (GB)	128	96/144
	Memory带宽	1.6TB/s	Up to 4TB/s
IO规格	UB Link	Up to 2016GB/s	2016GB/s
	PCIE	128GB/s RC & EP	128GB/s RC & EP
	RoCE/UBoE	400Gbps	400Gbps

硬件规格和组网如何影响优化措施

优化总览：需要**结合模型结构、硬件规格、集群拓扑**多维度考虑，差异化调整具体优化措施

950系列特性点变化	对整网优化措施影响
算力规格变化	感知较少，部分场景多流/分核计算可操作空间更大
CV算力比提升	感知较少，但影响算子耗时占比，性能瓶颈点发生变化
新的量化数据类型	影响较大，新的量化方案和量化接口
访存带宽提升	感知较少，性能瓶颈点略微发生变化
内存容量提升	单卡能放下的权重更多，并行策略发生变化
新的组网拓扑	通信受拓扑影响，并行策略发生变化
CCU	通信单元发生变化，新增额外通信配置选项
其他算子变化	调用接口可能发生变化

目录

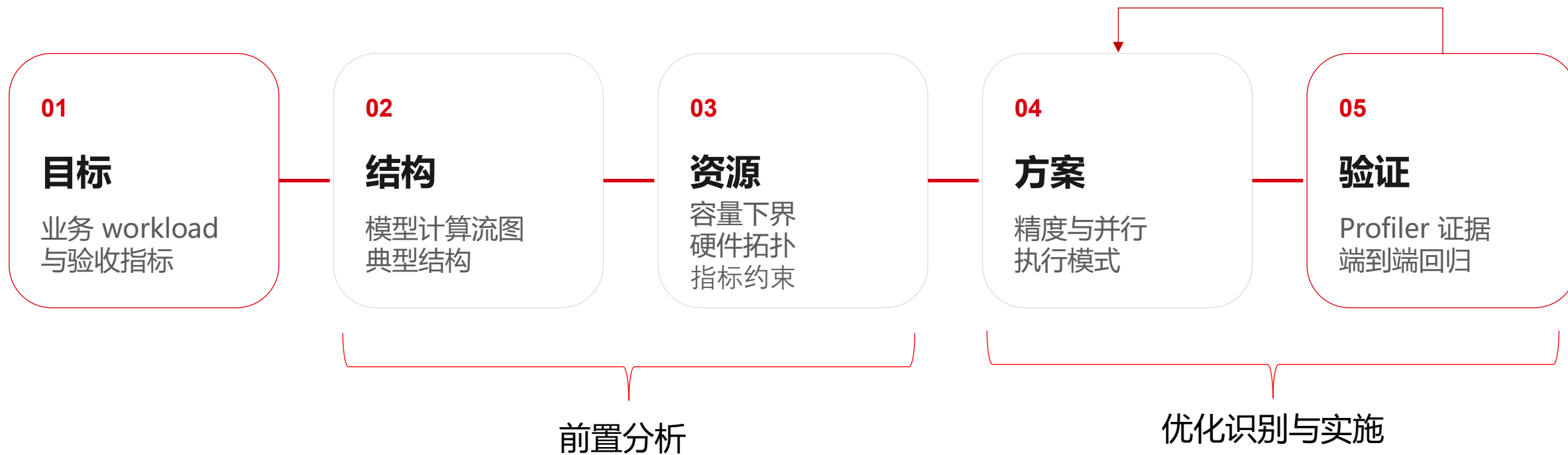
Part 1 Ascend 950硬件概览

Part 2 优化前置分析

Part 3 Qwen3.5优化点实施

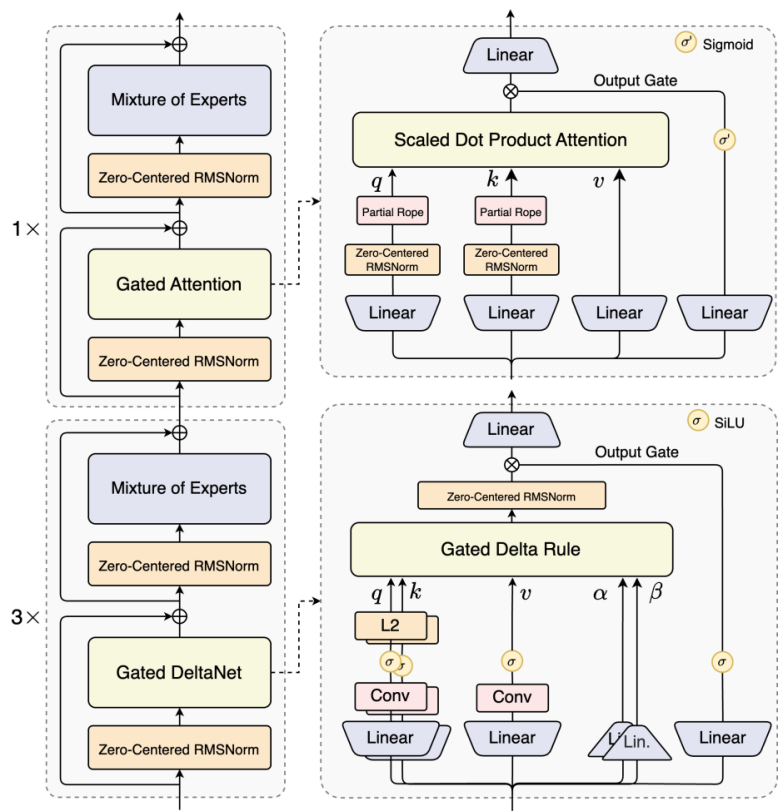
Part 4 总结

优化过程总览



优化前置分析 – 模型结构和规模

Q1：模型结构是什么样的？规模如何？



Qwen3.x系列模型结构一致：3 GDN + 1 Attention

配置	Qwen3-Next	Qwen3.5	Qwen3.6	Qwen3.7	Qwen3.8
代表最大模型	80B-A3B	397B-A17B	3.6-Max	3.7-Plus	2.4T-A95B
总参数量	80B	397B	未公开	未公开	2.4T
Activated Params	3B	17B	未公开	未公开	95B
Hidden Size	2048	4096	未公开	未公开	8192
Num Layers	48	60	未公开	未公开	92
Vocab Size	151,936	248,320	未公开	未公开	248,320
Architecture	Hybrid GDN + Attention	Hybrid GDN + Attention	同 Hybrid 家族	同 Hybrid 家族	Hybrid GDN + Attention
Linear Attention	Gated DeltaNet	Gated DeltaNet	Gated DeltaNet	Gated DeltaNet	Gated DeltaNet
Full Attention	Gated Attention	Gated Attention	Gated Attention	Gated Attention	Gated Attention
Layer Pattern	3 Linear + 1 Full	3 Linear + 1 Full	同	同	3 Linear + 1 Full

优化前置分析 – 模型结构和规模

Q1：模型结构是什么样的？规模如何？

单卡可用内存 ≥ 权重分片 + KV/递推状态 + 激活
/Workspace + 图缓存 + 余量

Qwen3.5模型	BF16 权重下界	FP8 权重下界	切分与精度类型
35B-A3B	约 70 GB	约 35 GB	EP8 / MXFP8 TP4
122B-A10B	约 244 GB	约 122 GB	TP8
397B-A17B	约 794 GB	约 397 GB	TP16

注：仅为参数量×字节数的理论权重下界，不代表实际部署占用。

不要用参数量直接承诺卡数；先给下界，再用峰值内存验证。

<https://gitcode.com/cann>

估算顺序

- 1 权重能否放下
- 2 状态/KV 是否主导
- 3 并发与长度峰值
- 4 Workspace 与图缓存
- 5 预留 10%–20%



优化前置分析 – 硬件规模下界

Q2: 有什么硬件? 规格如何? 规模如何?

单卡可用内存 ≥ 权重分片 + KV/递推状态 + 激活 /Workspace + 图缓存 + 余量

Qwen3.5模型	BF16 权重下界	FP8 权重下界	切分与精度类型
35B-A3B	约 70 GB	约 35 GB	EP8 / MXFP8 TP4
122B-A10B	约 244 GB	约 122 GB	TP8
397B-A17B	约 794 GB	约 397 GB	TP16

注: 仅为参数量×字节数的理论权重下界, 不代表实际部署占用。

不要用参数量直接承诺卡数; 先给下界, 再用峰值内存验证。

规格项		昇腾950PR	昇腾950DT
算力	Cube算力规格	378T (FP16/BF16)	432~486T (FP16/BF16)
		757T (MxFP8, HiF8, MxFP6)	864~972T (MxFP8, HiF8, MxFP6)
		1514T (MxFP4)	1728~1944T (MxFP4)
	Vector算力规格 (FP16/BF16)	47T	54~60.8T
	总算力规格 (FP16/BF16)	425T	486~547T
Memory	Memory容量 (GB)	128	96/144
	Memory带宽	1.6TB/s	Up to 4TB/s
IO规格	UB Link	Up to 2016GB/s	2016GB/s
	PCIE	128GB/s RC & EP	128GB/s RC & EP
	RoCE/UBoE	400Gbps	400Gbps

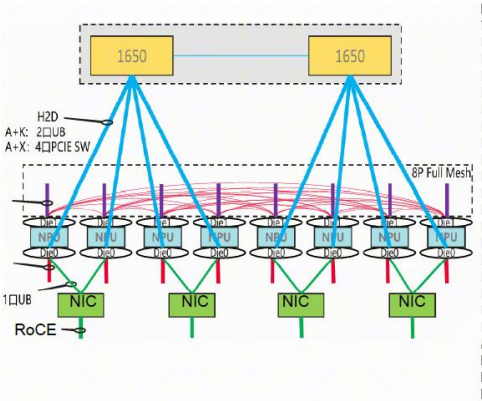
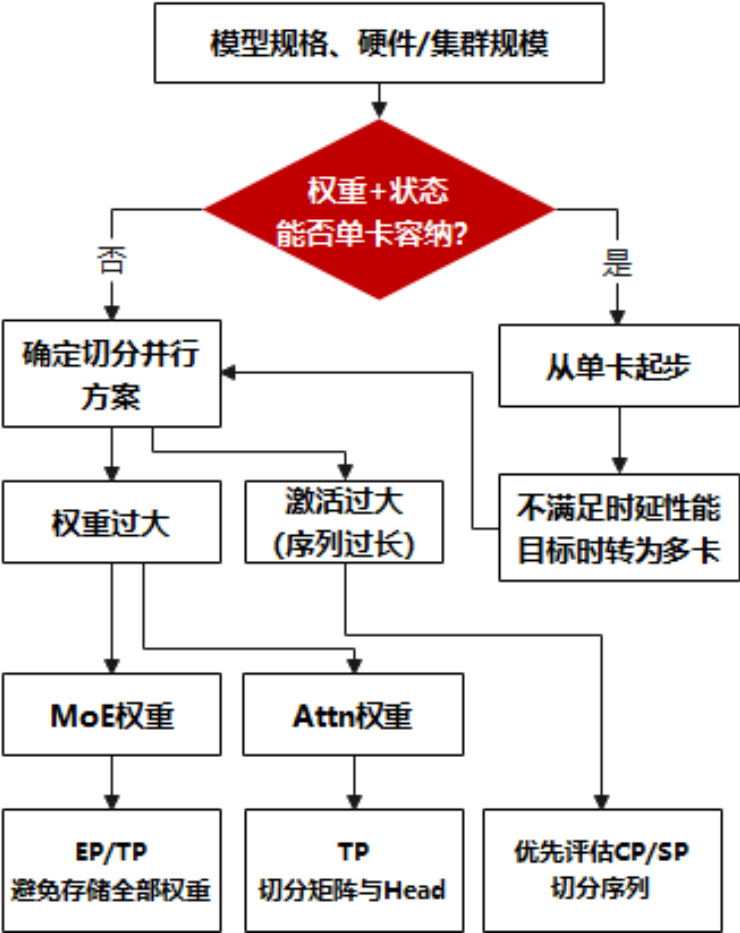


优化前置分析 – 组网和拓扑影响切分

Q3：有哪些可能的并行切分策略？

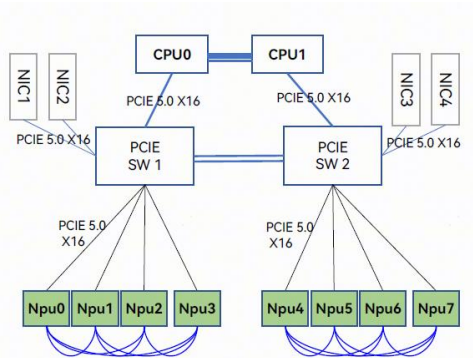
并行方案由“放得下”和“通信划算”共同决定

把并行组映射到拓扑，避免“卡数对了、性能错了”



Server形态

- ① 8 NPU + 2 CPU，CPU节点支持鲲鹏及X86
- ② server内：Fullmesh拓扑，7*X4 UB口
- ③ 出server：Clos拓扑，8*X4 UB口



标卡形态

- ① 基于X86底座，支持四卡UB互联
- ② 每卡3个X4 UB端口组成Fullmesh连接
- ③ 可接第三方网卡，做参数面互联方案

- TP** AllReduce 频繁，尽量不跨慢链路
- EP** All-to-All + 负载波动，关注专家分布与热点
- 跨机** 先测 HCCL 基线，再决定是否扩大并行组

目录

Part 1 Ascend 950硬件概览

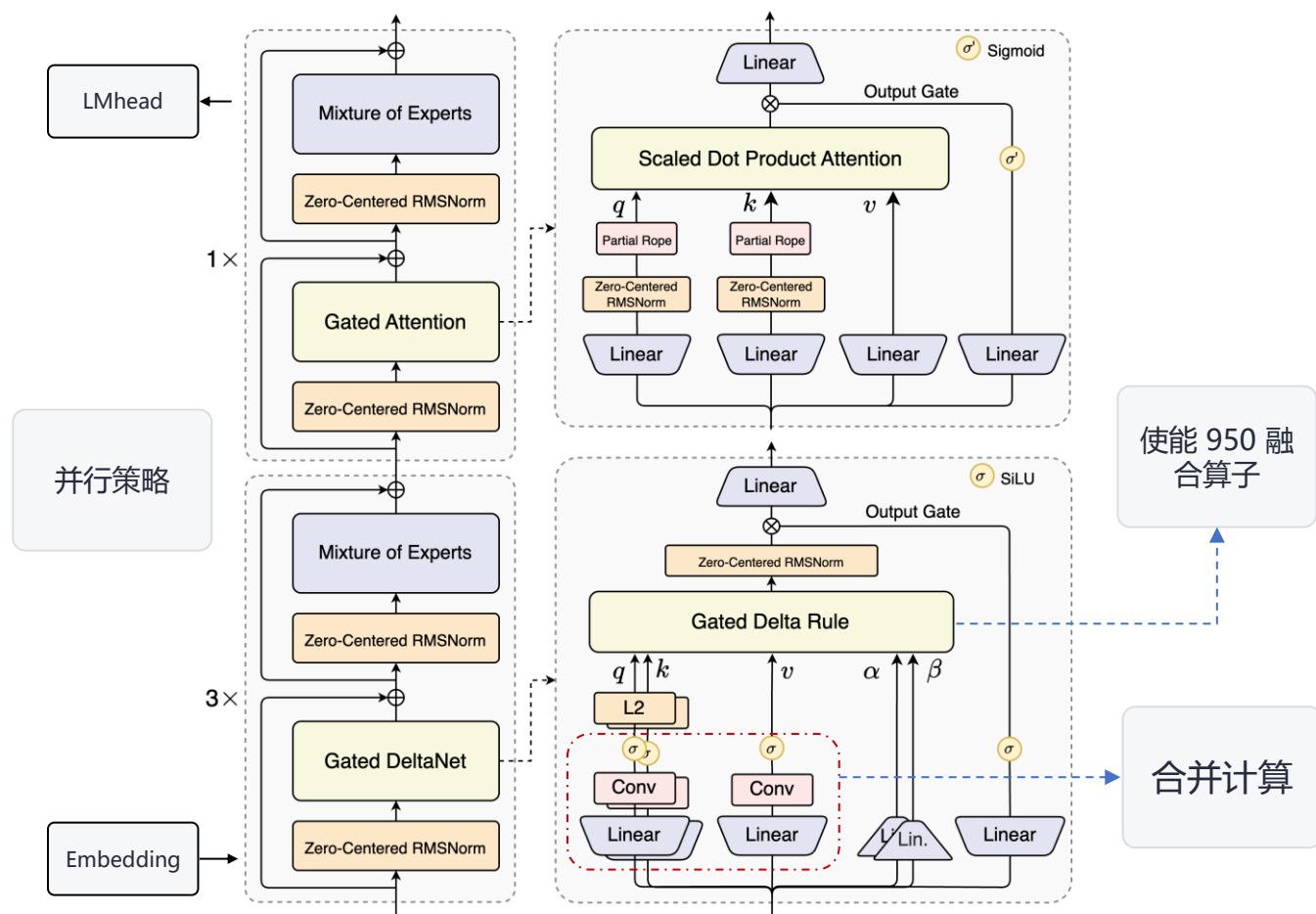
Part 2 优化前置分析

Part 3 Qwen3.5优化点实施

Part 4 总结

Qwen3.5优化措施

Qwen3.5同时包含Attention、Linear Attention和MoE，端到端性能受到单卡计算、数据搬运、多卡通信和框架调度共同影响，因此需要从多个维度协同优化。



融合算子

缩短单卡关键计算链路，识别Qwen3.5中的连续计算模式，将多个小算子合并执行，减少算子下发、中间Tensor读写和同步开销。

量化

将占据主要参数量和计算量的Linear、Projection或MoE Expert矩阵由BF16降低到FP8或MXFP8，提升低精度计算吞吐，减少权重显存和访存带宽压力。

并行策略与通信

根据模型不同模块的特征，区分不同的并行策略，从而平衡计算、显存和跨卡开销。在此基础上，再利用950 ccu通信能力优化并行产生的集合通信开销。

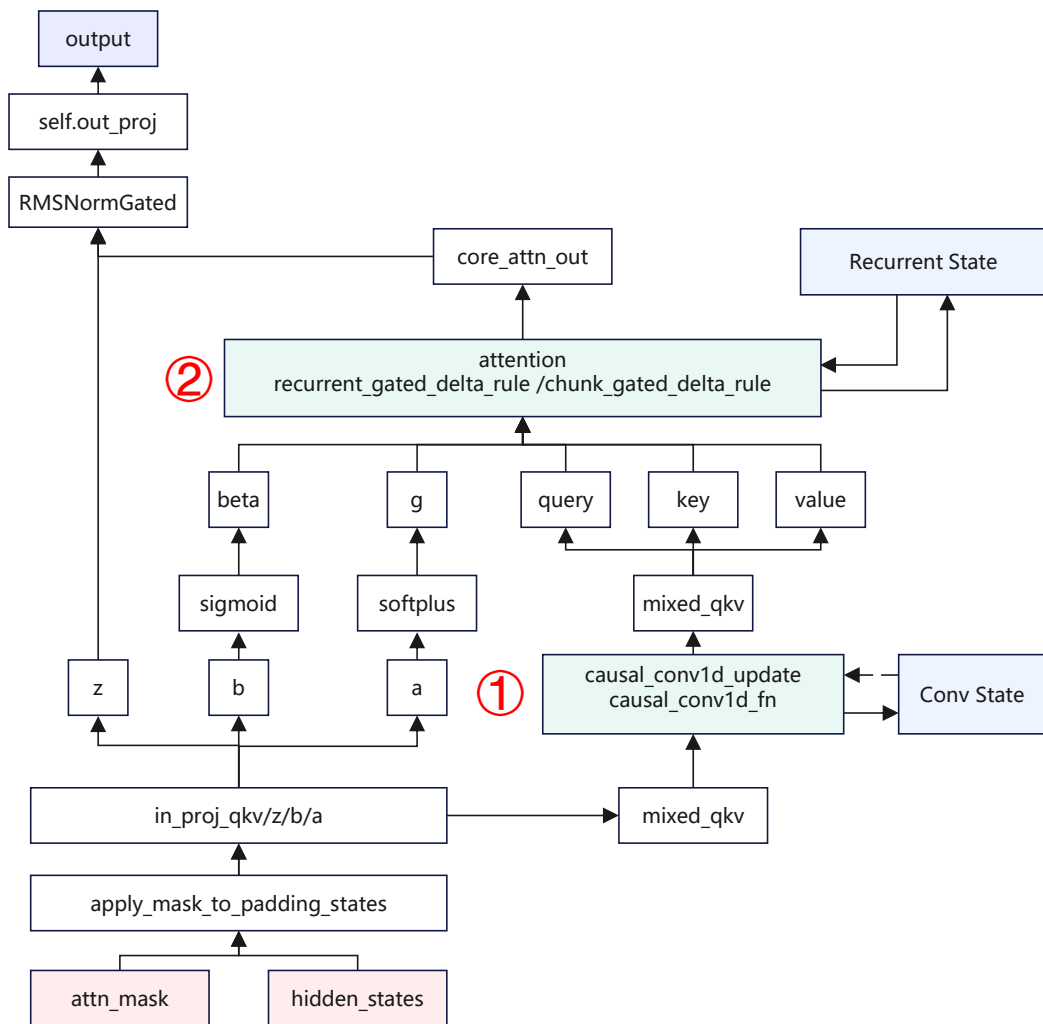
公共优化

图模式、权重合并、Weight NZ等优化不依赖单一芯片特性，可在910系列和950平台复用，用于减少框架调度、格式转换等开销。

除以上优化外，还可以继续评估多流并行、权重Prefetch、投机推理等方案；这些优化与业务场景、模型结构及部署规模强相关，本次不作为重点展开。

Qwen3.5关键计算链路识别

Gated DeltaNet 主链:



Gated DeltaNet 识别算子如下:

Prefill / Causal conv1d fn: ①

- 面向完整序列，采用因果 Padding 后执行 Conv1D。
- **状态初始化**：从输入末尾截取 $\text{kernel_size}-1$ 个元素

Decode / Causal conv1d update:

- 将历史 Conv State 与当前输入拼接成卷积窗口，无需 Padding。
- **状态滚动更新**：计算当前 Token 输出的同时，保留最近 $\text{kernel_size}-1$ 个输入。

Prefill • Chunk GDR: ②

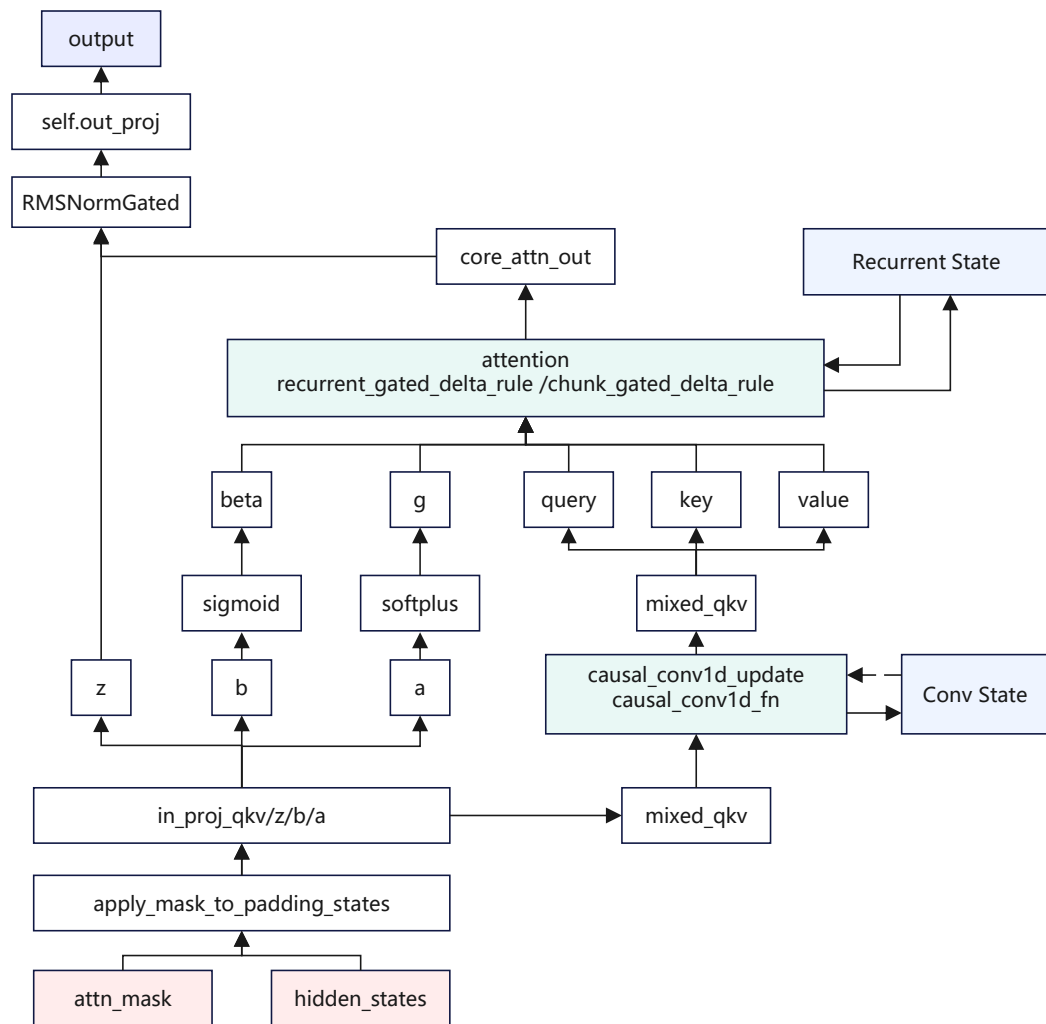
- 按 chunk 组织长序列计算，暴露块内并行。
- 减少通用递推产生的中间 Tensor 和逐步调度。

Decode • Recurrent GDR:

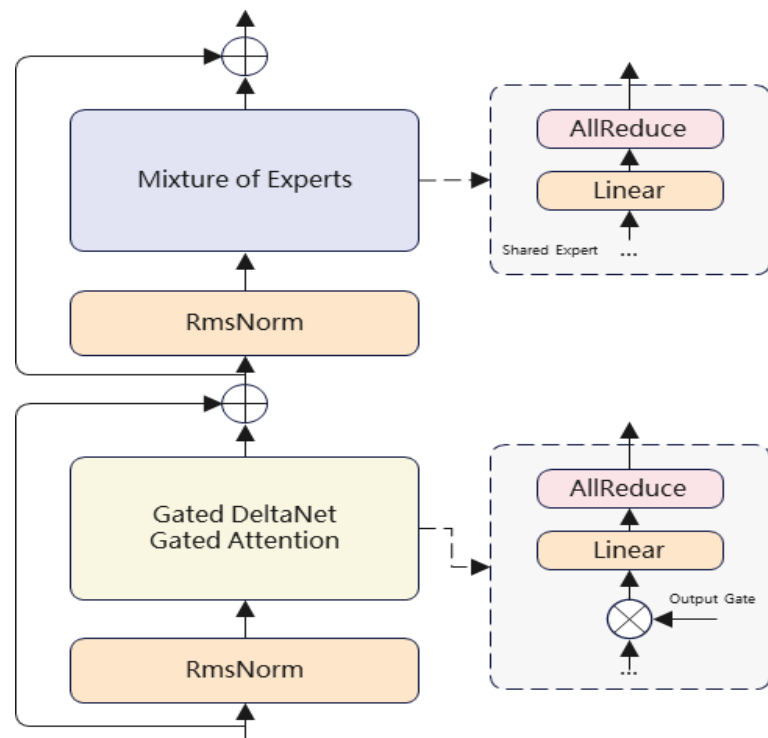
- 每步只消费新 token 和历史状态。
- 融合状态更新、门控与归一化，压缩热循环。

Qwen3.5关键计算链路识别

Gated DeltaNet 主链:



MM AllReduce 识别定位:



- 可识别到`recurrent_gated_delta_rule\chunk_gated_delta_rule\causal_conv1d_fn\causal_conv1d_update\mm_all_reduce`等融合算子需求。
- 根据已明确Qwen3.5模型侧识别的融合算子需求，进一步梳理各算子在950平台上的使能方法。

Qwen3.5 & 950融合算子使能

950 融合算子使能：

以recurrent_gated_delta_rule算子为例：

```
def torch_recurrent_gated_delta_rule(
    query, key, value, g, beta, initial_state, output_final_state, use_qk_l2norm_in_kernel=False
):
    initial_dtype = query.dtype
    if use_qk_l2norm_in_kernel:
        query = l2norm(query, dim=-1, eps=1e-6)
        key = l2norm(key, dim=-1, eps=1e-6)
    query, key, value, beta, g = [
        x.transpose(1, 2).contiguous().to(torch.float32) for x in (query, key, value, beta, g)
    ]

    batch_size, num_heads, sequence_length, k_head_dim = key.shape
    v_head_dim = value.shape[-1]
    scale = 1 / (query.shape[-1] ** 0.5)
    query = query * scale

    core_attn_out = torch.zeros(batch_size, num_heads, sequence_length, v_head_dim).to(value)
    last_recurrent_state = (
        torch.zeros(batch_size, num_heads, k_head_dim, v_head_dim).to(value)
        if initial_state is None
        else initial_state.to(value)
    )

    for i in range(sequence_length):
        q_t = query[:, :, i]
        k_t = key[:, :, i]
        v_t = value[:, :, i]
        g_t = g[:, :, i].exp().unsqueeze(-1).unsqueeze(-1)
        beta_t = beta[:, :, i].unsqueeze(-1)

        last_recurrent_state = last_recurrent_state * g_t
        kv_mem = (last_recurrent_state * k_t.unsqueeze(-1)).sum(dim=-2)
        delta = (v_t - kv_mem) * beta_t
        last_recurrent_state = last_recurrent_state + k_t.unsqueeze(-1) * delta.unsqueeze(-2)
        core_attn_out[:, :, i] = (last_recurrent_state * q_t.unsqueeze(-1)).sum(dim=-2)

    if not output_final_state:
        last_recurrent_state = None
    core_attn_out = core_attn_out.transpose(1, 2).contiguous().to(initial_dtype)
    return core_attn_out, last_recurrent_state
```

小算子版本

```
core_attn_out = torch_npu.npu_recurrent_gated_delta_rule(
    query.to(torch.bfloat16),
    key.to(torch.bfloat16),
    value.to(torch.bfloat16),
    last_recurrent_state.to(torch.bfloat16),
    beta=beta.to(torch.bfloat16),
    scale=scale,
    actual_seq_lengths=self.gdn_actual_seq_lengths,
    ssm_state_indices=self.gdn_ssm_state_indices,
    num_accepted_tokens=None,
    g=g.to(torch.float32),
    gk=None,
)
```

融合算子版本

融合算子通过减少算子下发与中间数据搬运、提升片上数据复用和计算流水效率，并结合硬件特性进行统一调度，从而缩短完整计算链路的整体开销。

上述收益能否真正兑现，取决于不同硬件平台对融合算子的支持程度。以下是这些融合算子在各个硬件设备上的支持情况：

模型关键链路	对应融合算子	算子支持情况
Prefill Causal Conv1D	causal_conv1d_fn	950
Decode Causal Conv1D	causal_conv1d_update	950
Prefill GDR	npu_chunk_gated_delta_rule	910系列/950
Decode GDR	npu_recurrent_gated_delta_rule	910系列/950
TP线性层	npu_mm_all_reduce_base	A2/950

950PR/DT 数据格式与 Qwen3.5 量化选择

为什么量化：降低大模型矩阵计算的数据成本

收益主要来源：

Qwen3.5的有大量参数和计算集中在矩阵乘。这些权重在每个请求、每个生成Token中都会被反复读取和计算。将适合量化的矩阵从BF16降低到8bit，主要带来三类收益：

- 提升矩阵计算吞吐
- 降低显存和带宽压力
- 提升端到端推理效率

量化原则：不同模块对量化误差的敏感程度不同，因此实际原则是：
优先量化收益占比高、算子支持完善、精度风险可控的矩阵；对敏感模块和收益较小的模块保留BF16或FP32。

950 支持数据格式

常规浮点

FP32 · FP16 · BF16

8 bit 浮点

HiF8 · FP8 E4M3FN/E5M2 · MXFP8

4 bit 浮点

MXFP4 · FP4 E2M1/E1M2

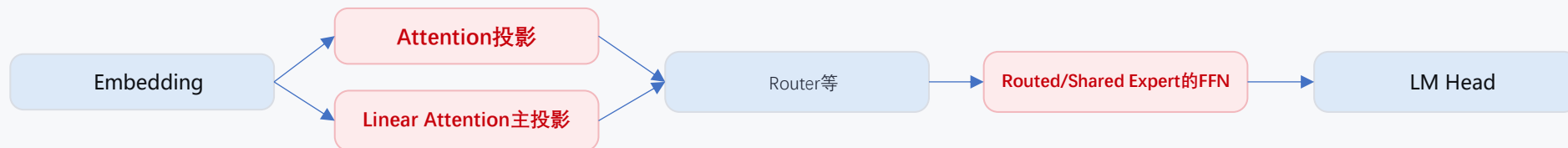
整数

INT8 · INT4

当前Qwen3.5量化方案

Qwen3.5在950上采用“主干矩阵低精度、敏感模块保留高精度”的选择性量化方案，提供**FP8**和**MXFP8**两条路径。

量化范围：



Qwen3.5 并行方案选择

为什么并行：在显存、计算粒度与通信之间寻找平衡

背景需求：

单卡部署面临多类约束：

- 模型权重、KV/SSM Cache和Workspace带来显存压力；
- 单卡需要承担全部矩阵计算，Prefill和大Batch场景计算量较大；

多卡并行可以分摊权重和计算，但也会引入通信与数据重排。因此目标不是“并行度越大越好”，而是：

“在显存可行的前提下，保持足够大的矩阵计算粒度，并尽量减少热路径上的跨卡通信”

情况一：对于Attention、Shared Expert、Embedding、LM Head等常规模块，需要在TP切分与权重复制之间权衡：

- 显存是否允许
- 计算粒度是否足够
- 通信开销是否值得

“核心：在显存可行的前提下，比较“TP的计算分摊收益”与“复制的完整矩阵及无通信收益”。”

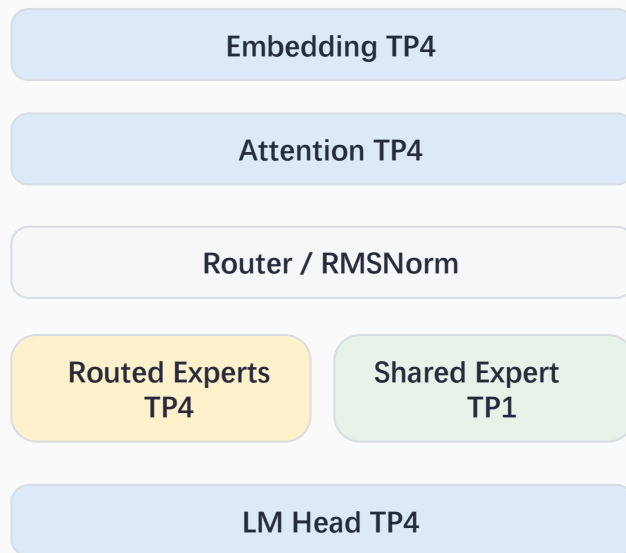
<https://gitcode.com/cann>

情况二：MoE模块需要在TP与EP之间做选择，二者对应不同的性能取舍。

依据：需综合比较负载均衡情况、矩阵粒度和通信开销，实际情况下，**Batch、输入长度、Expert规模、卡数和组网拓扑等发生变化后，最优方案都可能改变。**

Qwen3.5示例切分方案

当前Qwen3.5-35B采用以TP4为主的并行方案。该方案是当前卡数、Batch及序列配置下的实测选择，不代表其他部署Case的固定最优解。

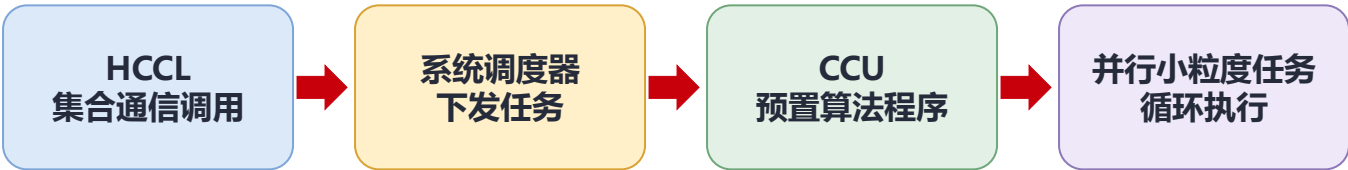


CANN

CCU 集合通信加速

多卡并行分摊了权重和计算，却不可避免的引了AllReduce和AllToAll等集合通信；因此并行策略确定后，还需要利用950的CCU能力优化通信执行。

通信算法程序在专用 CCU 上展开与执行



执行模式	主要特点
CCU_MS	使用片上 Memory Slice 中转，减少内存读写带宽
CCU_SCHED	由 CCU 负责通信调度，Rank 间直接进行内存数据传输

950DT侧重利用片上MS降低数据搬运开销，950PR通过CCU调度模式降低通信编排开销。

支持的集合通信原语

Broadcast ReduceScatter AllGather
AllReduce All2All All2AllV

Qwen3.5 接入

- hcccl_op_expansion_mode 控制 HCCL 通信域的展开模式。
- 多卡初始化时按950PR / 950DT自动设置。
- 模型侧保持标准 AllReduce / AllGather / ReduceScatter 调用。

常规 TP 通信使用 CCU；MoE MC2 使用独立 AIV 通信域

权重合并 & Weight NZ

完成算子、量化和通信等950专项优化后，可以通过跨平台公共优化进一步提升性能。

权重合并

合并把多次小矩阵读取和计算合成一次大 MatMul，结果再按逻辑维度拆分。

QKV合并示例：

分离： 3 次读取 $X \cdot 3$ 次 MatMul 下发

合并： 一次读取输入 · 一次较大 MatMul · 输出按视图拆分

分离路径

$X \times W_q$

$X \times W_k$

$X \times W_v$

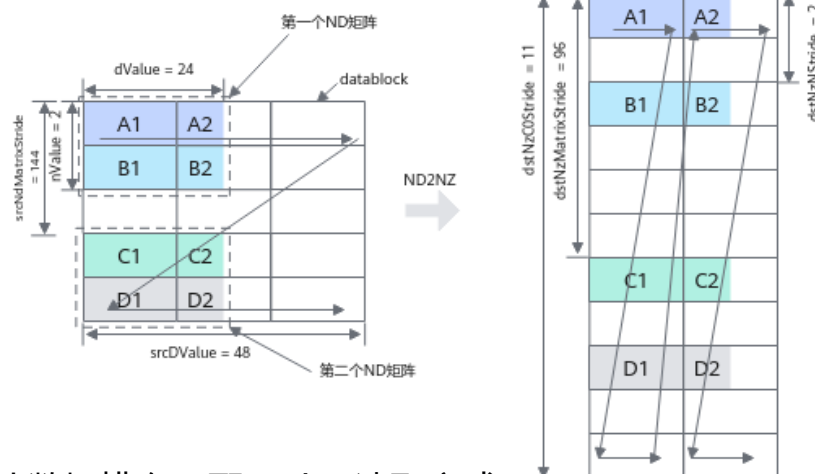
合并路径

$X \times [W_q | W_k | W_v]$

Weight NZ

ND: Checkpoint 的常规连续矩阵

NZ: 按 Cube tile 重排并补齐



<https://gitcode.com/cann>

NZ 让数据排布匹配 Cube 读取方式。

Qwen3.5 实际合并

- Full Attention: $Q + K + V$
- Shared Expert: Gate + Up
- Linear Attention: qkvz 与 ba 两组
- MoE Expert: Gate / Up 合并

ND NZ 执行路径对比

不预转换

ND Weight

每次 MatMul
随路 ND2NZ

Cube

提前转换

load once
format=29

NZ Weight

Cube

Weight NZ 不减少参数量，作用是让权重布局直接匹配 Cube 的分块读取方式，避免每次 MatMul 隐式执行 ND2NZ。

CANN

目录

Part 1 Ascend 950硬件概览

Part 2 优化前置分析

Part 3 Qwen3.5优化点实施

Part 4 总结

总结



每次优化都要回答：**瓶颈在哪里？改动为什么有效？收益在哪些边界内成立？**

cann-recipes-infer仓qwen3.5样例：https://gitcode.com/cann/cann-recipes-infer/tree/master/models/qwen3_5

一句话总结：**模型优化需综合考虑算法网络结构、硬件特性/规格与集群拓扑、应用场景与并发约束**
不能忽略任意一个维度

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.



关注cann-recipes-infer仓



关注CANN公众号获取资讯

<https://gitcode.com/cann>

CANN